



Chapter – 3

JSON : (Java Script Object Notation)





JSON is an acronym for JavaScript Object Notation, is an open standard format, which is lightweight and text-based, designed explicitly for human-readable data interchange. It is a language-independent data format. It supports almost every kind of language, framework, and library.

3.1 Concept and Features of JSON

Concepts

JSON is a lightweight text-based open standard data-interchange format. It is human readable. JSON is derived from a subset of JavaScript programming language (Standard ECMA-262 3rd Edition—December 1999). It is entirely language independent and can be used with most of the modern programming languages.

JSON is often used to serialize and transfer data over a network connection, for example between the web server and a web application. In computer science, serialization is a process to transforming data structures and objects in a format suitable to be stored in a file or memory buffer or transmitted over a network connection. Later on, this data can be retrieved. Because of the very nature of the JSON, it is useful for storing or representing semi structured data

JSON is a standard and is specified on RFC4627 on IETF (International Engineering Task Force). The specification is made by **Douglas Crockford** on July 2006.

JSON files are saved with **.json** extension. Internet media type of JSON is "application/json".

Features

- JSON is light-weight
- JSON is language independent which works with most of the modern days programming languages.
- It is easy to read and write
- Text-based, human readable data exchanged format
- It is scalable.

Some other features can be narrated as simplicity, openness, self-describing, internationalization, extensibility and interoperability.

3.2 Similarities and Differences between JSON and XML

JSON stands for JavaScript Object Notation, whereas XML stands for Extensive Markup Language. Nowadays, JSON and XML are widely used as data interchange formats, and both have been acquired by applications as a technique to store structured



data. The purpose of the comparison it's definitely not in the line of which is better, rather we will try to understand which one is suitable for storing specific kind of data.

Similarities between JSON and XML

- Both JSON and XML are human readable language.
- Both JSON and XML support unicode, thus any human written language can be written in JSON and XML documents.
- Both JSON and XML can be parsed.
- The data in both JSON and XML can be fetched using XMLHttpRequest.

Differences between JSON and XML

JSON (JavaScript Object Notation)	XML (eXtensible Markup Language)
JSON is simple and easier to read and write	XML is verbose and less readable
JSON doesn't use end tag	In XML, the end tag is mandatory
JSON supports array thus it is easy to transfer a big chunk of homogeneous data items using JSON	XML doesn't support array
JSON is easier to parse and can be parsed to ready-to-use JavaScript object	XML is difficult to parse than JSON
JSON is short	XML document is lengthy, verbose and redundant
JSON is less secure than XML	XML is more secured than JSON
JSON file is more readable than XML because it is short and to the point.	XML file is big and filled with user-defined tags, thus less-readable
JSON is data-oriented	XML is document-oriented
It doesn't provide display capabilities.	It provides the display capability because it is a markup language.
Example: ["student" : { "sname" : "Ashish", "rno" : "101", "div" : "I" },	 <student> <sname>Ashish</sname> <rno>101</rno> I </student>



```
{  
  "sname" : "Binita",  
  "rno" : "102",  
  "div" : "I"  
}  
]
```

3.3 JSON objects (with strings and numbers)

JSON objects are formed using the curly braces '{' which surrounds its data. These are written in a key-value pairing format. It is to be noted that the keys must have to be a string, and the value in the key-value pair must have to be anyone among the data types of JSON like: string, number, boolean, array, object, null. Moreover, the key and value are separated by a colon (:) and where there are multiple key-value pairs, all of these are separated by a comma (,)

Creating JSON object

There are 3 ways to create objects in JSON, the type of creation depends on how we create and initialize them. They are as follows:

- **Empty Object**

To create an empty JSON object the syntax is as follows:

Syntax:

```
var objname={};
```

Example:

```
var empobj={};
```

- **New Object**

Another way to create and object in JSON is as follows where you can use new operator along with object method.

Syntax:

```
var objname=new Object();
```

Example:

```
var jsonobj=new Object();
```

- **Object with attribute**

Another way of creating objects is by assigning multiple attributes where the key will be the string, and the value can be either string or a numeric value. Let us take an example where a JSON file will store students' data and their marks.



Example:

```
var students = {"name" : "Vishal", "IS" : 35, "IOT" : 42 }
```

Accessing JSON object value

To access the value of JSON object one can use the square bracket notations as follows for the above mentioned student object of JSON:

```
ismarks=students["IS"];
```

Nesting JSON object

To nest a JSON object within another one, we can use it like follows:

```
var students = {  
  "name" : "Vishal",  
  "course" : "BCA",  
  "marks": {  
    "IS" : 45;  
    "IOT" : 41;  
    "JAVA" : 35;  
  }  
}
```

Deleting JSON object

It is also possible to delete any properties of an object created within JSON using the delete keyword. The dot (.) or period operator needs to be used to access the nested object elements. The more you access the nested elements, the more you have to add dots. Let suppose you want to delete the "JAVA" from that previous JSON example so that you can use the delete keyword followed by this

```
delete students.marks.JAVA;
```

Creating JSON object (with string)

We can use the string as value for the object attribute. It is a double-quoted set of character(s) (Unicode) having backslash escaping. It denotes a single character string value, having a string length of one when you say character. Example of the same are already given in the example above with name, course attributes of JSON object students.

Creating JSON object (with numbers)

We can use the number as value for the object attribute. It provides floating point double precision data format and does not allow octal or hexadecimal number formats. Moreover, it does not assign NaN or infinity to its variables. It can be in 3 categories: Integer, Fraction, Exponent. Example of the same is given in the above JSON object named students with attributes IS, IOT, JAVA.



3.4 JSON Arrays and their examples

An array is a collection of multiple values under one name and is enclosed within square braces as in other programming languages and separated by commas. JSON array represents ordered list of values. JSON array can store multiple values. It can store string, number, boolean or object in JSON array. In JSON array, values must be separated by comma. The [] (square bracket) represents JSON array. An array of JSON can be array of strings, numbers, boolean, objects etc. Following are certain examples of JSON arrays:

3.4.1 Arrays of String

An array of string values can be represented as follows for the array days in JSON:

```
"days": ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday",  
         "Saturday"];
```

3.4.2 Arrays of Numbers

An array of number values can be represented as follows for the array in JSON:

```
"maxtempweek": [23, 24, 22, 22, 23, 24, 21];
```

3.4.3 Arrays of Boolean

An array of boolean values can be represented as follows for the array in JSON:

```
"rainydayweek": [true, false, false, true, true, false, true];
```

3.4.4 Arrays of Objects

JSON array can hold the JSON object as an array elements. Check it in the example below:

```
{ "students": [  
  { "name": "Manoj", "email": "mjpatel@gmail.com", "rno": 57 },  
  { "name": "Mansi", "email": "mansi124@gmail.com", "rno": 45 },  
  { "name": "Rivan", "email": "rivannaik@gmail.com", "age": 15 }  
]
```

3.4.5 Multi-dimensional Arrays

JSON multi-dimensional array works in following manner:

- **Creation**

An array of array or saving an array within other one is considered as multi-dimensional JSON array.



```
var webinfo = {  
  "name" : "blogger",  
  "users_auth" : [  
    [ "admins", "7", "7", "7"],  
    [ "editors", "4", "4", "1"],  
  ]  
}
```

- **Iteration**

To iterate through the multi-dimensional array in JSON simple for loop will work.

Code	Output
<pre>for (i in webinfo .users_auth) { for (j in webinfo.users_auth[i]) { x = webinfo.users_auth [i][j]; console.log(x); } }</pre>	<pre>admins 7 7 7 editors 4 4 4 1</pre>

3.5 JSON Comments

JSON is a data-only format. Comments in the form `//`, `#`, or `/* */`, which are used in popular programming languages, are not allowed in JSON. You can add comments to JSON as custom JSON elements that will hold your comments, but these elements will still be data. To do this, you need to add an element to your JSON file, such as `"_comment"`, which will contain your comment as a value of it. The JSON API endpoint must ignore this particular JSON comment element. In this JSON comment example, we have included two comment elements in the JSON data.

```
{  
  "Id": 1007,  
  "Customer": "Thomas",  
  "Quantity": 5,  
  "Price": 100.00,  
  "Date": "12-11-21",  
  "//first_comment": "Customer Bill.",  
  "//second_comment": "generated with 5 main attributes."  
}
```




Douglas Crockford, who popularized the JSON data format, deliberately removed comments from JSON to prevent misuse of the JSON format and keep it as a data-only format. He describes the reason he removed the comments from the JSON as follows:

” I removed comments from JSON because I saw people using them to store parsing directives, which would break compatibility.”

Therefore, the only option for adding comments to JSON is a workaround to use custom elements to store comments in a JSON file.

